

Introduction.

Risk-based testing (RBT) allocates effort based on the probability and impact of failure in each system component. Applying this strategy to a production TypeScript monorepo requires two inputs that are not available at planning time: observed defect data and real coverage measurements. The initial planning phase produces a risk model based on assumptions. The baseline automation study then delivers a first test suite and pipeline. This paper reports a third iteration. It closes the feedback loop by re-evaluating the model using empirical evidence, extending the suite to probe the original assumptions, and documenting what the plan missed.

The study covers the Circles project, a full-stack TypeScript monorepo. It includes a React frontend, a Hono REST API backend, and shared utility packages. The system is built and tested through Vite+, a unified toolchain that wraps Vite, Vitest, Oxlint, and Oxfmt under a single CLI. The study is guided by three research questions:

- RQ1** Which risk scores from the initial planning phase need revision based on empirical evidence?
- RQ2** What undocumented system behaviours are exposed by edge-case and failure-scenario tests?
- RQ3** How does the observed automation coverage compare with the targets set in the planning phase?

The paper contributes: (1) a revised risk matrix with evidence-backed score changes, (2) twelve new test cases across all mandatory categories, (3) documentation of three unexpected system behaviours, and (4) a comparison of planned versus actual QA outcomes.

Methodology.

Risk Re-evaluation.

The original risk formula is kept: Risk Score = Probability x Impact, with both axes scored 1 to 5. Three evidence sources are used to revise scores:

- (a) **Pipeline runs**: defects found, flaky test count, and per-module coverage from the baseline CI pipeline.
- (b) **Edge-case findings**: unexpected behaviours exposed by the new test cases.
- (c) **Coverage gap analysis**: modules at zero coverage get a higher score because detectability cannot be assessed without data.

Test Case Design.

New test cases target previously identified high-risk modules and are assigned to four categories:

Category	Goal
Failure scenarios	Verify correct error propagation on bad input or exhausted budget
Edge cases	Probe boundary and structural limits of parsing and validation
Concurrency	Confirm isolation of parallel invocations
Invalid input	Reject malformed or semantically incorrect data

Table 1: Test category mapping

All new tests use Vitest via the Vite+ toolchain, consistent with the baseline suite. Asynchronous tests use `vi.useFakeTimers()` to remove timing-dependent non-determinism. Cases come from reading the implementation. They target inputs that lie on or just outside the boundaries enforced by each function's guard conditions.

Quality Gate Evaluation.

Five quality gates defined in the baseline automation study are re-evaluated:

Gate	Metric	Threshold	Status
QG01	High-risk module coverage	$\geq 80\%$	Partial
QG02	Critical defects on trunk	0	Pass
QG03	Test suite execution time	≤ 5 min	Pass
QG04	Regression success rate	100 %	Pass
QG05	Static analysis violations	0 major	Pass

Table 2: Quality gate status

QG01 is partially met. The four automated modules each reach 100%. Three high-risk modules (API controllers, workflow orchestration, auth middleware) remain at 0%.

Preliminary Results.

Revised Risk Matrix.

Module	Baseline	Revised	Δ
Spotify OAuth & session mgmt	20	20	0
Data ingestion pipeline	20	22	+2
Dashboard stats & filtering	15	15	0
API endpoint correctness	12	14	+2
Friend feed & social features	9	9	0
Playlist management	6	6	0
AI discovery tools	6	6	0
Time Machine view	4	4	0
Static UI components	2	2	0

Table 3: Revised risk scores. Score = Probability $\times$ Impact (1–5 each).

Two scores increased. The **data ingestion pipeline** rises from 20 to 22 because the extended tests confirm two silent data-quality failure modes (see Section IV-C). The **API endpoint correctness** module rises from 12 to 14 because it remains at 0% coverage. Code review shows that the auth middleware throws HTTP 401 on missing session only, with no payload schema validation at the controller level.

Automation Evidence.

Test failures. No failures were observed in any CI run. All 33 tests pass on every push to trunk.

Flaky tests. None detected. Fake timers remove real-delay dependencies in all asynchronous tests.

Coverage. Four library modules reach 100% line coverage. Three integration-layer modules remain at 0%. Overall high-risk coverage is $4/7 = 57\%$, below the QG01 threshold of 80%.

Execution time. The backend unit suite completed in 206 ms after the suite extension. The baseline was 95 ms. The increase is proportional to the number of new tests and well within the 5 min gate.

Unexpected System Behaviours.

Three behaviours were not predicted by the initial risk model:

F-01 - Null album-artist bypass. `isValidEntry` does not check `master_metadata_album_artist_name`. Records with a null artist field pass validation and are ingested. This produces incomplete statistics on the dashboard without any error signal. Test TC-EXPORT-EDGE-01 confirms this.

F-02 - Permissive URI parser. `trackIdFromUri` extracts the third colon-delimited segment regardless of the resource type prefix. A Spotify episode URI (`spotify:episode:xyz`) yields an episode ID rather than a track ID. Test TC-EXPORT-INVAL-01 confirms this. Permissive-parser acceptance is a known production failure pattern in TypeScript validation layers.

F-03 - Two-byte ZIP check. `isZip` checks only `bytes[0] === 0x50 && bytes[1] === 0x4b` (the PK signature prefix). The ZIP specification requires a four-byte local file header (PK\x03\x04). Tests TC-ZIP-EDGE-01 through TC-ZIP-EDGE-03 confirm that two bytes are enough to pass the current guard.

These findings are not test failures. The code behaves as written. They are design decisions with unintended consequences that the initial risk model underestimated.

New Test Cases.

Test ID	Category	Scenario	Pass
TC-EXPORT-FAIL-01	Failure	<code>ms_played = -1 → false</code>	✓
TC-EXPORT-FAIL-02	Failure	No-colon URI → undefined	✓
TC-EXPORT-EDGE-01	Edge	Null <code>album_artist_name</code> → true (F-01)	✓
TC-EXPORT-EDGE-02	Edge	Empty string → undefined	✓
TC-EXPORT-EDGE-03	Edge	Base-62 ID round-trip	✓
TC-EXPORT-IN-VAL-01	Invalid input	Episode URI → permissive extract (F-02)	✓
TC-RETRY-FAIL-01	Failure	<code>retries=1, rate limit</code> → 1 attempt	✓
TC-RETRY-CONC-01	Concurrency	Two parallel calls resolve independently	✓
TC-RETRY-IN-VAL-01	Invalid input	Non-Error rejection → immediate rethrow	✓
TC-ZIP-EDGE-01	Edge	2-byte PK array → true (F-03)	✓
TC-ZIP-EDGE-02	Edge	1-byte array → false	✓
TC-ZIP-EDGE-03	Edge	1 024-byte PK-prefixed array → true	✓

Table 4: Extended test cases. All 12 pass.

Metrics Summary.

Metric	Planning	Baseline	Extended
Total automated tests	9	21	33
Backend suite time	–	95 ms	206 ms
Test pass rate	100 %	100 %	100 %
Flaky test rate	–	0 %	0 %
CI pipeline pass rate	100 %	100 %	100 %
High-risk module coverage	15 %	67 %	57 %
Critical defects found	0	0	0
Unexpected behaviours found	–	0	3

Table 5: QA metrics across study phases

The overall high-risk coverage decreases from 67% to 57% between the baseline and the extended study. This is not a regression. The denominator expands because three previously untracked integration-layer modules (API controllers, workflow orchestration, auth middleware) are now included in scope.

Comparative Analysis.

Planned vs Actual.

Aspect	Planned	Observed
Unit coverage target	$\geq 80\%$ on <code>src/lib/</code>	100 % on 4 modules; 0 % on 3 others
E2E coverage	Auth, dashboard, API flows	2 tests — home page only
API integration tests	Planned in first iteration	Not implemented after two iterations
CI pass rate	100 % on every PR	100 % (met)
Flakiness rate	$< 5\%$ over 30 runs	0 % (exceeded target)
Execution time	< 5 min	206 ms (well within target)
Risk model accuracy	9 modules scored	2 revised; 3 new behaviours found
Total effort	~40 hours	~55 hours across all phases

Table 6: Planned vs observed QA outcomes

Incorrect Assumptions.

Three planning assumptions from the initial risk study did not hold.

Integration tests were assumed straightforward. Controller-level tests depend on Better Auth session handling. This requires a live database or a carefully constructed mock. This complexity was underestimated, and both subsequent iterations deferred this work.

The entry validator was assumed complete. `isValidEntry` was expected to validate all fields needed for accurate statistics. Tests TC-EXPORT-EDGE-01 and TC-EXPORT-INVAL-01 show that `album_artist_name` is not checked and non-track URI types are not rejected.

ZIP validation was assumed standard. The implementation was expected to check the full four-byte local file header. It checks only two bytes (F-03).

Missing Scenarios and Design Gaps.

Two structural weaknesses are identified in the current automation design.

Backend integration tests. The pure-function suite provides a stable foundation. Backend integration tests against a dedicated test database are already in development. They will extend coverage to HTTP handler behaviour and database interaction without requiring a live deployment.

E2E coverage. The Playwright configuration, CI workflow, and dashboard E2E suites are already in place. The groundwork is done. Expanding to the OAuth callback and file import flows is straightforward from here.

Discussion.

The risk-first automation strategy worked well for the library layer. Targeting four high-risk pure functions first produced a stable, fast, and deterministic suite in 206 ms. The Vite+ toolchain removed per-file configuration and ensured identical test behaviour in CI and local environments.

The edge-case expansion uncovered three design-level observations (F-01, F-02, F-03) that the initial risk model did not predict. The null-field bypass (F-01) and permissive-parser acceptance (F-02) both support raising the data-ingestion risk score from 20 to 22.

The primary outstanding gap is controller-level integration coverage. The next iteration introduces backend integration tests that run against a real test database. This closes the gap between pure-function coverage and deployed behaviour. Playwright suites for the dashboard flow are already in development. They extend beyond the two existing home-page tests to cover dashboard rendering and the file import flow.

QG01 (coverage $\geq 80\%$) remains unmet at 57%. The threshold is not too strict. It correctly identifies the gap. The failure is due to insufficient test scope, not poor code quality. QG02 through QG05 all pass, confirming that the automated scope is stable and the pipeline functions correctly.

Specific improvements for the next iteration: restrict `trackIdFromUri` to `spotify:track:*` URIs; add `album_artist_name` validation to `isValidEntry`; strengthen `isZip` to verify all four header bytes; consolidate extended spec files with the baseline spec files.